

# Milestone 3: Chunked Prefill + Radix Prefix Cache

Sarosh Khan and Naomie Chien

5/26/2026

**Environment** Model: Qwen/Qwen3-8B GPU: L4

All paged runs use `--mem-fraction-static 0.85 --page-size 32`.

## 1 Design and Implementation

### 1.1 Part A: Chunked Prefill

Milestone 2 processed every admitted request’s full prompt in a single packed varlen forward (`flash_attn_varlen_func`). At high concurrency the combined Q-token count can run into the tens of thousands, and the resulting activation tensors (attention scores, SwiGLU intermediates) can exceed L4 SRAM before any output is generated, causing a CUDA OOM that kills the whole batch.

The critical change is `_prefill_batch_chunked` in `engine.py`, which replaces the single-shot prefill with an iterative loop. Each scheduler step feeds at most `prefill_chunk_size` tokens per in-flight request, accumulates the resulting KV pages in the pool via `_store_prefill_kv_to_pool`, and defers sampling until the full prompt is consumed. This bounds per-step activation memory to a fixed budget regardless of prompt length or concurrency. The scheduler exposes this as `--prefill-chunk-size N`; `N=0` keeps the Milestone-2 single-shot path.

When a request has a cached prefix from Part B, the cached tokens are skipped before applying the chunk cap, so the two features compose without extra bookkeeping.

We use `--prefill-chunk-size 512`. A much smaller budget multiplies kernel-launch overhead without reducing peak memory meaningfully; a much larger one re-introduces OOM risk once several requests are mid-prefill simultaneously. At 512 tokens per step the L4 stays comfortable at concurrency 4, and at the benchmark’s mean input of 759 tokens each request fires two chunks ( $\lceil 759/512 \rceil = 2$ ), keeping scheduling overhead small.

### 1.2 Part B: Radix Prefix Cache

We implemented `RadixCache` in `miniengine/radix_cache.py`, a radix tree mapping page-aligned token prefixes to lists of physical KV pages. Matching walks the tree from the root along the longest stored token sequence and returns a `MatchResult` at the deepest page-aligned boundary; partial-page tails are never returned, preserving the attention metadata invariants. After each request completes, the scheduler inserts the prompt plus generation tokens so subsequent multi-turn requests that resend conversation history also hit on the prior turn’s response.

The two other critical pieces are eviction and pinning. Each node carries a `last_access` timestamp; when `KVMemoryPool.allocate` exhausts the free list it calls `cache.evict(n)` to reclaim the  $n$  least-recently-used unlocked pages, so cached and free pages share the same physical budget with no double-counting. `inc_lock_ref` / `dec_lock_ref` on the matched node pin it for the lifetime of

the borrowing request so eviction cannot recycle a page that is actively being attended over. The flag `--disable-radix-cache` turns the cache off; it is on by default.

## 2 Performance

### 2.1 Benchmarking Setup

All runs use Qwen/Qwen3-8B on an L4 GPU with `--mem-fraction-static 0.85 --page-size 32`. Part A compares `--prefill-chunk-size 0` (single-shot baseline) against `--prefill-chunk-size 512`. Part B compares the radix cache (default) against `--disable-radix-cache` at fixed `--prefill-chunk-size 512`.

- **bench\_serving**: WildChat-style prompts, closed-loop at concurrencies 1, 4, 16; reports TTFT, TPOT, completion latency, and GenTok/s.
- **bench\_accuracy**: 200 MMLU samples, max-tokens 32, concurrency 8.
- **bench\_cache --workload shared**:  $K$  groups  $\times$   $N$  questions with a shared prefix; reports wall time, throughput, TTFT, and cache hit rate.
- **bench\_cache --workload multiturn**:  $S$  sessions  $\times$   $T$  turns; reports the same metrics plus a per-turn breakdown.

### 2.2 Part A: Chunked Prefill

#### OOM demonstration

We launched the server with `--prefill-chunk-size 0` and ran **bench\_serving** at concurrencies 16 and 32.

Listing 1: Server (unchunked) and client invocation

```
python3 -m miniengine --model Qwen/Qwen3-8B --mode paged \
--mem-fraction-static 0.85 --page-size 32 --prefill-chunk-size 0

python3 -m benchmark.bench_serving --conurrencies 16,32
```

Listing 2: bench\_serving outcome, chunk-size 0

| Conc | TTFT_p50<br>(ms) | TTFT_p99<br>(ms) | Compl_p50<br>(ms) | Compl_p99<br>(ms) | TPOT_p50<br>(ms) | TPOT_p99<br>(ms) | GenTok/s | OK |
|------|------------------|------------------|-------------------|-------------------|------------------|------------------|----------|----|
| 16   | ALL FAILED       |                  |                   |                   |                  |                  |          |    |
| 32   | ALL FAILED       |                  |                   |                   |                  |                  |          |    |

All requests fail. The server log shows CUDA OOM as the combined prefill activation tensors for 16 or more concurrent long prompts exhaust L4 VRAM. Restarting with `--prefill-chunk-size 512`:

Listing 3: Server (chunk-size 512) and client invocation

```
nohup python3 -m miniengine --model Qwen/Qwen3-8B --mode paged \
--mem-fraction-static 0.85 --page-size 32 --prefill-chunk-size 512 \
> ~/chunked-server.log 2>&1 &

python3 -m benchmark.bench_serving --conurrencies 4,8
```

Listing 4: bench\_serving outcome, chunk-size 512, concurrency 4

```
Running concurrency=4 (16 requests, closed-loop)...
```

|                                                                                   |    |         |         |              |        |                |                  |
|-----------------------------------------------------------------------------------|----|---------|---------|--------------|--------|----------------|------------------|
| [ 1/16]                                                                           | ok | in=1525 | out=0   | ttft=        | 0ms    | compl=12179ms  | (elapsed 12.2s)  |
| [ 2/16]                                                                           | ok | in=1971 | out=2   | ttft=12178ms |        | compl=16124ms  | (elapsed 16.1s)  |
| [ 3/16]                                                                           | ok | in=1582 | out=2   | ttft=15996ms |        | compl=16961ms  | (elapsed 17.0s)  |
| [ 4/16]                                                                           | ok | in=1577 | out=0   | ttft=        | 0ms    | compl= 9492ms  | (elapsed 26.5s)  |
| ...                                                                               |    |         |         |              |        |                |                  |
| [13/16]                                                                           | ok | in=2038 | out=151 | ttft=        | 4090ms | compl=16839ms  | (elapsed 104.9s) |
| [14/16]                                                                           | ok | in=1688 | out=1   | ttft=15996ms |        | compl=600038ms | (elapsed 600.0s) |
| [15/16]                                                                           | ok | in=1463 | out=16  | ttft= 3434ms |        | compl=600611ms | (elapsed 627.1s) |
| [16/16]                                                                           | ok | in=1322 | out=0   | ttft=        | 0ms    | compl=600936ms | (elapsed 634.0s) |
| conc=4 TTFT p50=4708ms p99=15691ms Compl p50=12179ms p99=22950ms GenTok/s=6 ok=13 |    |         |         |              |        |                |                  |

  

| Conc | TTFT_p50   | TTFT_p99 | Compl_p50 | Compl_p99 | GenTok/s | OK |
|------|------------|----------|-----------|-----------|----------|----|
| 4    | 4708       | 15691    | 12179     | 22950     | 6        | 13 |
| 8    | ALL FAILED |          |           |           |          |    |

The chunked server completes 13/16 requests at concurrency 4 without a server crash. The three timeouts and all failures at concurrency 8 come from KV-page pool exhaustion rather than activation OOM: chunked prefill bounds the per-step activation footprint but does not extend the physical page budget. The server stays alive and returns valid responses for the requests that finish; the unchunked run crashed the whole batch.

## Accuracy

Table 1: MMLU accuracy (200 samples, max-tokens 32, concurrency 8).

| Configuration            | Accuracy | Correct / Samples | Avg latency |
|--------------------------|----------|-------------------|-------------|
| --prefill-chunk-size 0   | 58.0%    | 116 / 200         | 0.93 s      |
| --prefill-chunk-size 512 | 59.0%    | 118 / 200         | 0.94 s      |

The two-sample difference is within the statistical noise of a 200-sample run; chunked prefill does not change model outputs.

## Serving benchmark at concurrency 1 / 4 / 16

Listing 5: bench\_serving concurrency sweep, chunk-size 512

```
Input lengths: min=517, max=1029, mean=759 tokens
Output lengths: min=259, max=504, mean=372 tokens
```

|         |                                 |            |                  |             |             |        |
|---------|---------------------------------|------------|------------------|-------------|-------------|--------|
| conc=1  | TTFT p50=304ms                  | p99=1638ms | TPOT p50=77.2ms  | p99=149.6ms | GenTok/s=13 | ok=8/8 |
| conc=4  | TTFT p50=482ms                  | p99=1420ms | TPOT p50=109.0ms | p99=176.1ms | GenTok/s=16 | ok=5/8 |
| conc=16 | ALL FAILED (KV-pool exhaustion) |            |                  |             |             |        |

With `--prefill-chunk-size 0`, concurrencies 16 and 32 OOM as shown above; at concurrencies 1 and 4 the activation budget is not saturated on the L4 at this input length, so the unchunked path produces latency within noise of the chunked path. At concurrency 4, five of eight requests finish within the 600 s timeout and three stall on KV-page allocation. TPOT rises from 77 ms to 109 ms as decode batches grow, consistent with the throughput-latency tradeoff in batched inference.

Table 2: `bench_serving` default workload, `--prefill-chunk-size 512`. TTFT and TPOT in ms.

| Conc | TTFT <sub>p50</sub>                   | TTFT <sub>p99</sub> | TPOT <sub>p50</sub> | TPOT <sub>p99</sub> | GenTok/s | OK  |
|------|---------------------------------------|---------------------|---------------------|---------------------|----------|-----|
| 1    | 304                                   | 1638                | 77.2                | 149.6               | 13       | 8/8 |
| 4    | 482                                   | 1420                | 109.0               | 176.1               | 16       | 5/8 |
| 16   | ALL FAILED (KV-pool exhaustion on L4) |                     |                     |                     |          |     |

## 2.3 Part B: Radix Prefix Cache

### Shared workload, fixed config

Config: 10 groups  $\times$  9 questions (10 $\times$ 10 occasionally failed on the final SSE chunk; 10 $\times$ 9 was stable), `--shared-prefix-len 2000`, `--concurrency 1`.

Listing 6: `bench_cache` shared workload, cache ON

```
python3 -u -m benchmark.bench_cache --workload shared \
--num-groups 10 --questions-per-group 9 \
--shared-prefix-len 2000 --concurrency 1 --base-url http://localhost:8000
```

```
=====
Workload      : shared      Requests   : 90
Wall time     : 649.87 s    Throughput : 0.14 req/s
Gen tok/s     : 7
TTFT p50 / p99 : 2476 ms / 7622 ms
Cache hit rate : 87.6%
=====
Per-group breakdown:
group_0: 9 reqs prompt=21024 hit=18432 hit_rate=87.7% TTFT_p50=2473ms
group_1: 9 reqs prompt=21032 hit=18432 hit_rate=87.6% TTFT_p50=2480ms
... (all 10 groups within noise of each other)
```

Listing 7: `bench_cache` shared workload, cache OFF

```
python3 -u -m benchmark.bench_cache --workload shared \
--num-groups 10 --questions-per-group 9 \
--shared-prefix-len 2000 --concurrency 1 \
--disable-radix-cache --base-url http://localhost:8000
```

```
=====
Workload      : shared      Requests   : 90
Wall time     : 850.26 s    Throughput : 0.11 req/s
Gen tok/s     : 4
TTFT p50 / p99 : 6388 ms / 6528 ms
Cache hit rate : 0.0%
=====
Per-group breakdown:
group_0: 9 reqs prompt=21024 hit=0 hit_rate=0.0% TTFT_p50=6394ms
group_1: 9 reqs prompt=21032 hit=0 hit_rate=0.0% TTFT_p50=6442ms
... (all groups uniform, no reuse)
```

Table 3: Shared workload, `--shared-prefix-len 2000`: cache on vs. off.

| Config    | Wall time     | Throughput    | GenTok/s      | TTFT <sub>p50</sub>            | TTFT <sub>p99</sub> | Hit rate |
|-----------|---------------|---------------|---------------|--------------------------------|---------------------|----------|
| Cache ON  | 649.87 s      | 0.14 req/s    | 7             | 2476 ms                        | 7622 ms             | 87.6%    |
| Cache OFF | 850.26 s      | 0.11 req/s    | 4             | 6388 ms                        | 6528 ms             | 0.0%     |
| Speedup   | 1.31 $\times$ | 1.27 $\times$ | 1.75 $\times$ | <b>2.58<math>\times</math></b> |                     |          |

TTFT drops  $2.58\times$  (6388 ms to 2476 ms), exceeding the  $2\times$  target. The 87.6% hit rate comes from page-aligned matching: each request of roughly 2336 tokens shares 2304 cached tokens (72 pages at 32 tokens each), leaving one partial page to recompute. Token throughput improves  $1.75\times$ ; wall-time speedup is more modest at  $1.31\times$  because generation for the 64-token output cap still dominates under single concurrency.

## Prefix-length sweep

We ran `bench.cache --workload shared` (10 groups  $\times$  9 questions, cache ON) at four prefix lengths.

Table 4: Cache-on shared workload across prefix lengths (TTFT in ms).

| Prefix len | TTFT <sub>p50</sub> | TTFT <sub>p99</sub> | Hit rate | Wall time | Throughput |
|------------|---------------------|---------------------|----------|-----------|------------|
| 200        | 321 ms              | 1504 ms             | 74.6%    | 449.11 s  | 0.20 req/s |
| 500        | 674 ms              | 2155 ms             | 83.7%    | —         | —          |
| 2000       | 2476 ms             | 7622 ms             | 87.6%    | 649.87 s  | 0.14 req/s |
| 4000       | 4868 ms             | 27403 ms            | 88.1%    | 652.19 s  | 0.14 req/s |

Hit rate rises from 74.6% to 88.1% as prefix length grows, because more tokens fall within page-aligned boundaries. At prefix 200, each warm request skips 224 cached tokens (7 pages); at prefix 4000 it skips 4096 tokens (128 pages), an  $18\times$  increase in avoided prefill work. Cache-off TTFT scales roughly linearly with prompt length, so the absolute savings grow with prefix length: at prefix 200 the cold-miss TTFT is roughly 1504 ms while warm hits take 321 ms; at prefix 2000 the comparison is 6388 ms vs. 2476 ms. The ratio flattens at very long prefixes as page-load time becomes the floor, but the absolute prefill work avoided keeps increasing.

## Multi-turn workload

Config: 16 sessions  $\times$  8 turns, `--max-tokens 64`, `--concurrency 1`.

Listing 8: Multi-turn benchmark, cache ON

|                                                       |
|-------------------------------------------------------|
| =====                                                 |
| Workload: multiturn Requests: 128 Wall time: 439.06 s |
| Throughput: 0.29 req/s Gen tok/s: 12                  |
| TTFT p50 / p99: 386 ms / 661 ms Cache hit rate: 75.8% |
| =====                                                 |
| Per-turn breakdown:                                   |
| turn 0: hit_rate= 0.0% TTFT_p50= 100ms                |
| turn 1: hit_rate=57.3% TTFT_p50= 226ms                |
| turn 2: hit_rate=65.4% TTFT_p50= 294ms                |
| turn 3: hit_rate=87.8% TTFT_p50= 385ms                |
| turn 4: hit_rate=81.4% TTFT_p50= 392ms                |
| turn 5: hit_rate=81.8% TTFT_p50= 427ms                |
| turn 6: hit_rate=84.7% TTFT_p50= 494ms                |
| turn 7: hit_rate=86.9% TTFT_p50= 547ms                |

Listing 9: Multi-turn benchmark, cache OFF

|                                                       |
|-------------------------------------------------------|
| =====                                                 |
| Workload: multiturn Requests: 128 Wall time: 522.62 s |
| Throughput: 0.24 req/s Gen tok/s: 13                  |
| TTFT p50 / p99: 170 ms / 892 ms Cache hit rate: 0.0%  |
| =====                                                 |
| Per-turn breakdown:                                   |

```

turn 0: hit_rate= 0.0% TTFT_p50= 101ms
turn 1: hit_rate= 0.0% TTFT_p50= 106ms
turn 2: hit_rate= 0.0% TTFT_p50= 122ms
turn 3: hit_rate= 0.0% TTFT_p50= 162ms
turn 4: hit_rate= 0.0% TTFT_p50= 176ms
turn 5: hit_rate= 0.0% TTFT_p50= 226ms
turn 6: hit_rate= 0.0% TTFT_p50= 853ms
turn 7: hit_rate= 0.0% TTFT_p50= 873ms

```

Table 5: Multi-turn workload summary: cache on vs. off.

| Config    | Wall time | Throughput | GenTok/s | TTFT <sub>p50</sub> | TTFT <sub>p99</sub> | Hit rate |
|-----------|-----------|------------|----------|---------------------|---------------------|----------|
| Cache ON  | 439.06 s  | 0.29 req/s | 12       | 386 ms              | 661 ms              | 75.8%    |
| Cache OFF | 522.62 s  | 0.24 req/s | 13       | 170 ms              | 892 ms              | 0.0%     |

Table 6: Per-turn TTFT<sub>p50</sub> (ms): cache on vs. off.

| Turn | Cache ON | Cache OFF | Speedup     |
|------|----------|-----------|-------------|
| 0    | 100      | 101       | 1.0×        |
| 1    | 226      | 106       | 0.5×        |
| 2    | 294      | 122       | 0.4×        |
| 3    | 385      | 162       | 0.4×        |
| 4    | 392      | 176       | 0.4×        |
| 5    | 427      | 226       | 0.5×        |
| 6    | 494      | 853       | <b>1.7×</b> |
| 7    | 547      | 873       | <b>1.6×</b> |

Turn 0 is identical across both configurations. From turn 1, the cache begins absorbing prior-turn KV pages and hit rate climbs from 57.3% to 86.9% by turn 7. At turns 6 and 7, the cache-off engine must process the full growing conversation history (over 8000 tokens), pushing TTFT to 853–873 ms, while the cached engine holds at 494–547 ms, a 1.6–1.7× TTFT reduction. Aggregate throughput improves by 21% (0.24 to 0.29 req/s) and wall time drops by 16%. The aggregate TTFT<sub>p50</sub> is higher for cache-on (386 vs. 170 ms) because the median is pulled down by early turns where the cached prefix is short and cache lookup adds overhead; the per-turn breakdown shows how the benefit grows as history accumulates.

## Regression test

We ran `bench_serving` (WildChat workload) with the cache on and off to check for regressions on low-sharing traffic.

Listing 10: Regression test, concurrency 1, cache ON

```

conc=1 TTFT p50=227ms p99=1511ms Compl p50=11912ms p99=17704ms GenTok/s=13 ok=8/8

```

Listing 11: Regression test, concurrency 1, cache OFF

```

conc=1 TTFT p50=202ms p99=1460ms Compl p50=11580ms p99=17485ms GenTok/s=13 ok=8/8

```

Listing 12: Regression test, concurrency 4, cache OFF

```

conc=4 TTFT p50=272ms p99=2122ms Compl p50=14343ms p99=18635ms GenTok/s=18 ok=5/8

```

Table 7: bench\_serving regression: cache on vs. off, `--prefill-chunk-size 512`. TTFT and Compl in ms.

| Config    | Conc | TTFT <sub>p50</sub> | TTFT <sub>p99</sub> | Compl <sub>p50</sub> | Compl <sub>p99</sub> | GenTok/s | OK   |
|-----------|------|---------------------|---------------------|----------------------|----------------------|----------|------|
| Cache ON  | 1    | 227                 | 1511                | 11912                | 17704                | 13       | 8/8  |
| Cache OFF | 1    | 202                 | 1460                | 11580                | 17485                | 13       | 8/8  |
| Cache OFF | 4    | 272                 | 2122                | 14343                | 18635                | 18       | 5/8* |

\*Requests 6–8 timed out at conc=4; cache-on showed the same pattern.

At concurrency 1, cache-on and cache-off are within noise: TTFT<sub>p50</sub> differs by 25 ms and GenTok/s is identical at 13. At concurrency 4, requests 6–8 time out under both configurations due to L4 KV-page pressure, the same as Item 3 above, not a cache-induced regression.

### 3 Next Steps

The main bottleneck we found is KV-page pool exhaustion at moderate-to-high concurrency. Chunked prefill prevents activation OOM but does not address the fact that all in-flight requests continue to hold their decode pages indefinitely: once the pool fills, new prefills stall and existing decodes can OOM on the next page allocation. On the L4 with Qwen3-8B this starts to bite around concurrency 4 for the longer WildChat prompts (mean 759 tokens) and consistently at concurrency 16 for both unchunked and chunked modes.

A natural next step is **decode-time retraction**: when the pool cannot satisfy the next decode allocation, the scheduler picks a victim from the running set, evicts its pages back to the pool, and re-queues it for re-prefill when capacity returns. This trades temporary throughput for liveness and would let the engine serve concurrency 16 workloads that currently time out. The radix cache interacts with retraction in a useful way: a retracted request whose prefix is still cached does not need to redo the full prefill, only the suffix past the last cached page.

On the cache side, the multiturn hit rate stays in the 57–87% range rather than approaching 100% because the per-turn generation length is short (max 64 tokens) relative to the question overhead per turn, limiting how much of the next turn’s prompt is already cached. Longer generation budgets or a larger `--turns-per-session` would shift that ratio and drive the per-turn hit rate higher sooner, producing aggregate TTFT reductions above the 50% target for more of the turn range.